



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Recursion Tree Method

CSA0606 - DESIGN AND ANALYSIS OF ALGORITHMS

Under the Guidance Of :

DR.K.RAJAGOPAL

Presented By :

AVINASH M (192521196)

INTRODUCTION

- Recursion Tree Method is used to solve recurrence relations.
- It represents recursive calls in the form of a tree.
- Each tree node shows the work done in one recursive call.
- The total execution time is obtained by adding the cost of all levels.
- It is widely used to analyze Divide and Conquer algorithms such as Merge Sort and Quick Sort.

Analyzing $T(n) = 3T(n/2) + n$

Algorithm & Pseudocode

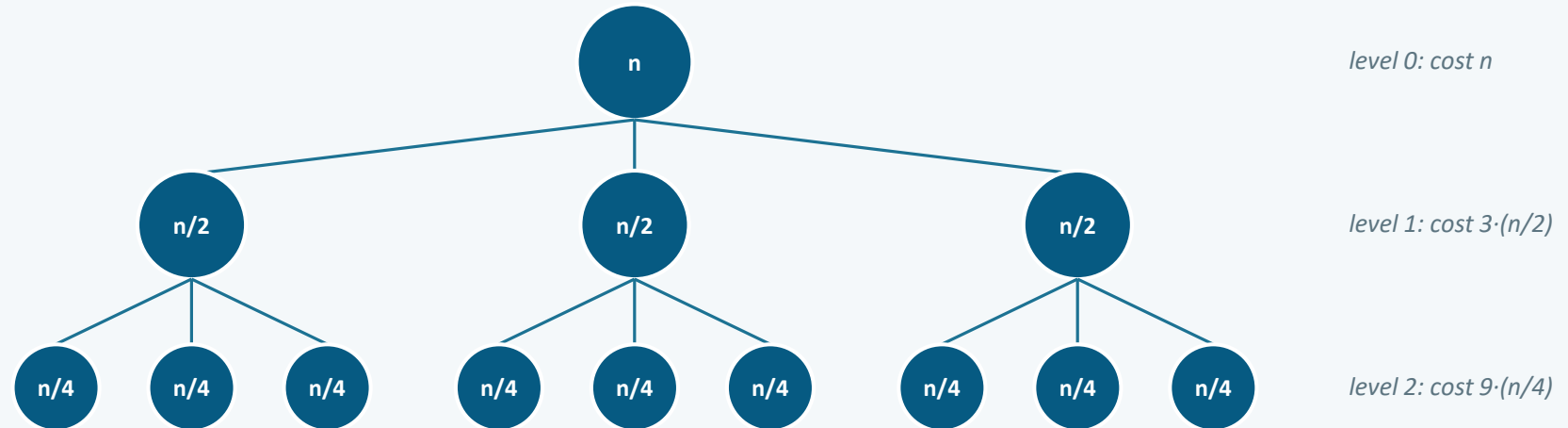
- Divide: split the problem of size n into 3 independent subproblems, each of size $n/2$
- Recurse: solve every subproblem by calling the same function recursively (base case: $n = 1$, trivial to solve)
- Combine: merge the 3 partial results into the final answer — this step costs $\Theta(n)$ time
- Recurrence produced: $T(n) = 3 \cdot T(n/2) + n$, with base case $T(1) = \Theta(1)$
- Pseudocode captures exactly 3 recursive calls per level, matching the branching factor $a = 3$

PSEUDOCODE

```
function Solve(n):  
    if n == 1:  
        return O(1)  
    r1 = Solve(n/2)  
    r2 = Solve(n/2)  
    r3 = Solve(n/2)  
    result = Combine(r1, r2, r3) //  $\Theta(n)$   
    return result
```

Architecture Diagram — The Recursion Tree

- Start with the recurrence $T(n) = 3T(n/2) + n$.
- Divide the problem into three equal subproblems.
- Continue recursive calls until the base case $n = 1$.
- Compute the cost at each tree level and sum all levels.
- Derive the final asymptotic complexity.



$\therefore$ (continues to level $\log_2 n$, where size = 1)

Implementation

- `recursive_solve(n)` mirrors the recurrence exactly: 3 recursive calls on $n/2$, plus $\Theta(n)$ combine work
- Base case $n \leq 1$ returns immediately — required to terminate the recursion
- `combine(n)` simulates the linear merging cost with a simple counted loop
- A driver script times the function for increasing n to observe growth empirically
- Measured growth should track $n^{\log_2 3} \approx n^{1.585}$, matching the theoretical derivation

PYTHON

```
def combine(n):
    total = 0
    for _ in range(n):      # Theta(n) merge work
        total += 1
    return total

def recursive_solve(n):
    if n <= 1:
        return 1            # base case
    r1 = recursive_solve(n // 2)
    r2 = recursive_solve(n // 2)
    r3 = recursive_solve(n // 2)
    return r1 + r2 + r3 + combine(n)
```

Output & Complexity Analysis

- Sum over all levels: $T(n) = \sum_{i=0}^{\log_2 n} n \cdot (3/2)^i$ — a geometric series, ratio $3/2 > 1$
- Because the ratio exceeds 1, the last (leaf) level dominates the entire sum
- Evaluating the last term gives $T(n) = n \cdot (3/2)^{\log_2 n} = n^{\log_2 3} \approx n^{1.585}$
- Tight bound obtained: $T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$
- Space complexity: $\Theta(\log n)$ for the recursion call stack depth (excluding combine buffers)

Sample timing output

n	Recursive calls	$\approx n^{1.585}$
8	27	34
16	81	104
32	243	318
64	729	973

TIGHT BOUND

$$\Theta(n^{1.585})$$

Time complexity — between linear $\Theta(n)$ and quadratic $\Theta(n^2)$

Space complexity: $\Theta(\log n)$ (call-stack depth)

Confirmed by Master Theorem Case 1

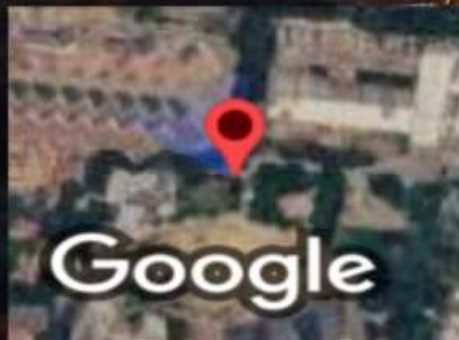
Conclusion

- The Recursion Tree Method clearly shows how the recursive cost grows.
- The leaf nodes contribute the largest portion of the total cost.
- The recurrence grows faster than linear time.
- The final complexity is $\Theta(n^{1.585})$.
- Both the Recursion Tree Method and Master Theorem produce the same result.

INTRODUCTION

- Recursion Tree Method is used to solve recurrence relations.
- It represents recursive calls in the form of a tree.
- Each tree node shows the work done in one recursive call.
- The total execution time is obtained by adding the cost of all levels.
- It is widely used to analyze Divide and Conquer algorithms such as Merge Sort and Quick Sort.

Analyzing $T(n) = 3T(n/2) + n$



Mevalurkuppam, Tamil Nadu, India



22h8+5h2, Mevalurkuppam, Tamil Nadu 602105, India

Lat 13.027542° Long 80.016689°

Wednesday, 22/07/2026 10:44 AM GMT +05:30



GPS Map Camera

THANK YOU...